

Situación Didáctica

TecLag Software avanza en el desarrollo de su proyecto del compilador del lenguaje **PaytonTK**, el equipo de ingenieros de software han implementado la etapa del análisis sintáctico y continúan ahora con el análisis semántico.

La Junta Directiva instruye al equipo de desarrollo a diseñar la etapa de análisis semántico como un comprobador de tipos estático, es decir, deberá ejecutar la comprobación de tipos en tiempo de compilación.

Usando la gramática de lenguaje PaytonTK diseñar un **Esquema de Traducción** que realice la comprobación de tipos en las sentencias del lenguaje aplicando las siguientes

Reglas de compatibilidad de tipos:

1. Todos los identificadores que se declaren se debe registrar su tipo de dato en la tabla de símbolos. Las constantes denotadas por **"num"**, **"num.num"** son de tipo `int` y `float` respectivamente. Las constantes **"literal"** son de tipo `string`.
2. No se permite la redeclaración de un identificador ya sea con el mismo tipo o diferente.
3. En la asignación se permite asignar una expresión entera o real a un identificador real, de lo contrario deben ser del mismo tipo. Una literal solo se puede asignar a un identificador tipo `string`.
4. Cuando en una sentencia se usa un identificador que no fue declarado se debe mostrar en el mensaje de error el nombre de ese identificador.
5. En expresiones condicionales se permite comparar expresiones numéricas de diferentes tipos, es decir, se puede comparar una expresión real con una expresión entera. Una literal solo se puede comparar contra otra literal o contra un identificador de tipo `string`.
6. Los nombres de funciones también deben tener tipo en la tabla de símbolos usando la Expresión de Tipo correspondiente. De igual forma todos los argumentos de funciones deben tener tipo en la tabla de símbolos.
7. Se debe realizar la comprobación de tipos al invocar una función comprobando el número y tipo de dato de los argumentos. Considerar además el tipo de dato que devuelve la función si es que su llamado forma parte de una expresión. El lenguaje permite invocar una función dentro de una expresión por ejemplo en :

$$2 * suma (a, b) + 1$$

en este caso el tipo devuelto por *suma* debe considerarse para determinar el tipo resultante de la expresión. Además de comprobar que los tipos de *a* y *b* sean compatibles con los argumentos definidos en la función *suma*.

Nota: El lenguaje PaytonTK tiene una limitante en su diseño que obliga a que todas **las variables declaradas y los argumentos de funciones** tengan nombres únicos, es decir, no se pueden repetir aunque estén declaradas en distinto ámbito.

Nota: Todas las variables deben estar declaradas con su tipo de dato antes de usarse.

Presentar el Esquema de Traducción como en el ejemplo de la pag. 32 de las notas del curso.